

Resource Allocation System

Iteration 32 – Shared-Room Exam Best-Fit Strategy

Najib Attar

May 2026

1 Iteration Goal

The goal of Iteration 32 is to add a new optional allocation strategy for shared-room exam allocation. Earlier multi-room exam strategies allowed one exam to be split across several classrooms, but they treated a classroom as fully blocked once any overlapping allocation existed in that room.

This iteration introduces `SharedRoomExamBestFitStrategy`. The new strategy allows different splittable exam requests to share the same physical classroom at the same overlapping time, but only when the total assigned participants in that classroom does not exceed the room capacity.

This behavior is added as a separate strategy. It does not replace or modify the existing strategies.

2 Changes from Previous Iteration

Iteration 31 extended the user and organization model. Iteration 32 focuses on allocation policy by adding a more flexible exam-sharing strategy.

The main changes are:

- Added `SharedRoomExamBestFitStrategy.h`.
- Added `SharedRoomExamBestFitStrategy.cpp`.
- Updated `AllocationStrategyFactory` to support `shared_room_exam_best_fit`.
- Updated `DataController` config validation to accept `shared_room_exam_best_fit`.

- Updated `data/config.txt` so the new strategy can be selected.
- Updated the Flask dashboard strategy dropdown to include Shared-Room Exam Best-Fit.
- Updated project documentation to mention the new strategy.
- Added demo data with room S401 and two overlapping splittable exam requests.

3 Scenario

Some universities may allow different exams to share the same classroom if the exams happen at the same time and the room has enough seating capacity. For example, if a classroom has 100 seats, one exam with 60 students and another exam with 40 students can share the room safely.

Before this iteration, `multi_room_exam_best_fit` blocked the whole room after the first exam allocation. Therefore, the second exam could not use the same classroom, even if enough seats remained.

In Iteration 32, the new shared-room strategy calculates the remaining capacity of the room and allows exam-to-exam sharing when capacity is still available.

4 Modules and Their Tasks

Module	Task in Iteration 32
<code>SharedRoomExamBestFitStrategy</code>	Implements the new exam-to-exam shared-room allocation policy. It processes normal requests using greedy behavior and applies shared-room best-fit only to splittable exams.
<code>AllocationStrategyFactory</code>	Returns <code>SharedRoomExamBestFitStrategy</code> when the config value is <code>shared_room_exam_best_fit</code> .
<code>DataController</code>	Accepts the new strategy name during config loading and validation.
<code>Allocation</code>	Continues storing <code>assignedParticipants</code> , which is required for checking shared room capacity.
<code>ExamRequest</code>	Provides exam-specific data, including <code>canSplitAcrossRooms</code> and the exam time slot.
<code>Space/Classroom</code>	Provides classroom capacity, type, building, status, and feature data used during candidate filtering.
<code>Flask Dashboard</code>	Displays the new strategy option in the strategy dropdown. It does not implement allocation logic.

5 Shared-Room Exam Best-Fit Design

The new strategy keeps the existing design principle that strategies decide allocation algorithms. It does not move allocation logic into the rule engine, data controller, or web dashboard.

For normal request types, the strategy delegates to the existing greedy behavior:

- `OneTimeRequest` is processed normally.
- `RecurringRequest` is processed normally.
- Non-splittable `ExamRequest` is processed like a normal one-room exam.
- `InvalidRequest` is skipped as a rejected record.

Only splittable `ExamRequest` objects use the shared-room capacity logic.

For a splittable exam, candidate classrooms must satisfy the normal exam constraints:

- the space type must be `Classroom`;
- the room must be available;
- the required building must match if specified;
- the required feature must match if specified;
- the requester must be allowed to use classrooms;
- the requester must be allowed to submit exam requests.

The key difference is how overlapping allocations are handled. An overlapping exam allocation no longer blocks the room immediately. Instead, the strategy calculates remaining room capacity:

$$remainingCapacity = room.capacity - \sum assignedParticipants$$

where the sum includes overlapping exam allocations in the same room.

The classroom may be used only if:

$$remainingCapacity > 0$$

The strategy must always preserve the safety condition:

$$totalAssignedParticipants \leq room.capacity$$

If an overlapping allocation belongs to a non-exam request, the room is treated as unavailable. This prevents mixing exams with meetings, labs, lectures, or other normal requests.

6 Policy Comparison

Strategy	Same-room sharing	exam	Multi-room splitting	exam	Capacity usage
multi_room_exam_best_fit	No		Yes		Uses full room as blocked after assignment
shared_room_exam_best_fit	Yes, only	exam-to-exam	Yes		Uses remaining classroom capacity

Table 2: Comparison of strict and shared-room exam allocation policies

Normal requests are not allowed to share rooms in either strategy. The new sharing behavior is limited to overlapping splittable exam requests and remains capacity-limited.

7 Iteration Comparison

The previous best-fit exam strategy selected rooms based on full room capacity. If a room already had an overlapping allocation, it was removed from the candidate list.

The new shared-room strategy uses available capacity chunks instead of full room capacities. For example, if S401 has 100 seats and 60 seats are already assigned to an overlapping exam, then the available capacity chunk is 40 seats.

The strategy still uses best-fit selection:

- minimize unused capacity first;
- if tied, prefer fewer rooms;
- keep deterministic behavior based on candidate order when needed.

This is not a global optimization algorithm. Requests are still processed in loaded order. The strategy only chooses a best-fit room combination for the current splittable exam using the currently available shared capacity.

8 Updated Class Diagram

The class diagram for this iteration adds `SharedRoomExamBestFitStrategy` as another concrete implementation of `IAAllocationStrategy`. It also shows that `AllocationStrategyFactory` can create or return this strategy. The new strategy uses `ExamRequest`, `Space`, `Classroom`, `Allocation`, and `assignedParticipants` to calculate remaining shared capacity.

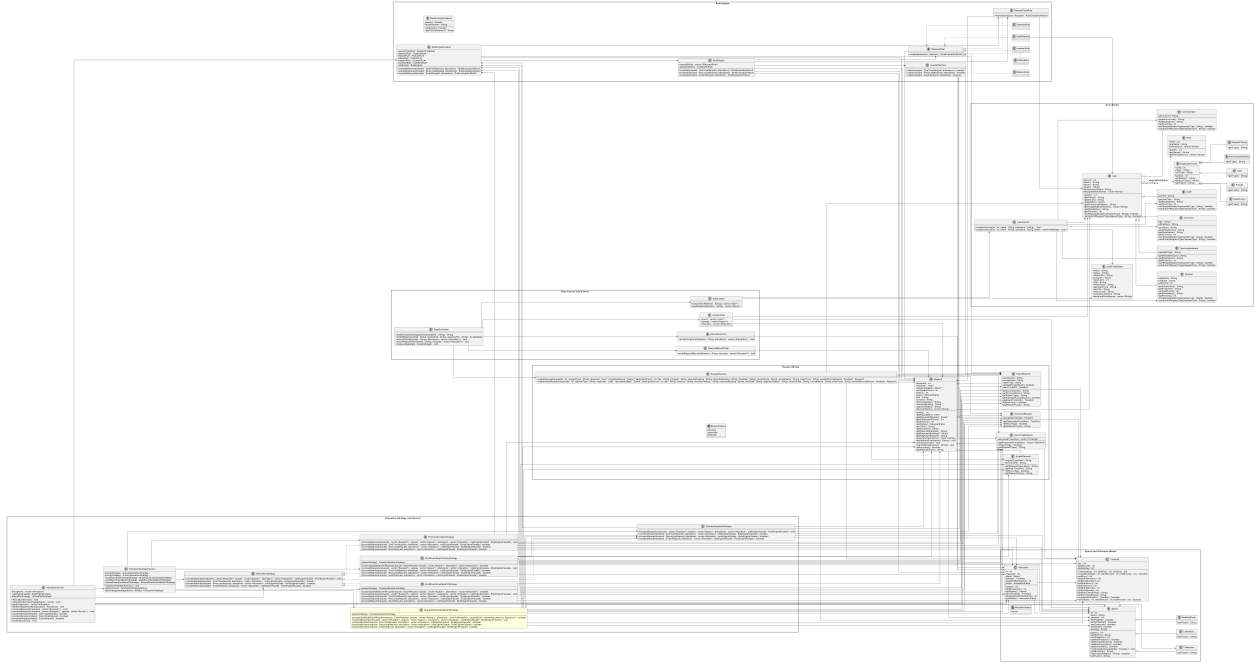


Figure 1: Updated class diagram for Iteration 32

9 Updated Workflow Diagram

The workflow remains similar to previous strategy-based iterations. The main difference is the splittable exam path when the selected strategy is `shared_room_exam_best_fit`.

The workflow includes:

- loading the selected strategy from `config.txt`;
- delegating request processing through `AllocationService`;
- processing non-splittable or non-exam requests normally;
- finding candidate classrooms for splittable exams;
- computing remaining capacity from overlapping exam allocations;

- blocking rooms with overlapping non-exam allocations;
- selecting a best-fit available capacity combination;
- committing allocations only if the full exam can be satisfied.

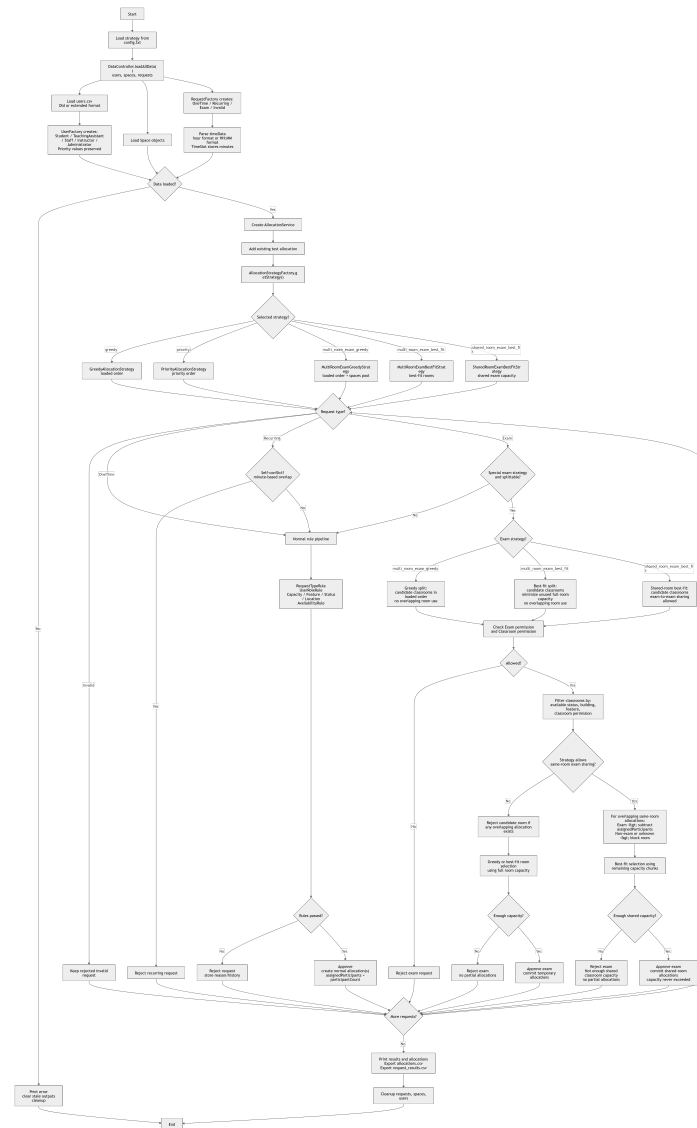


Figure 2: Updated workflow diagram for Iteration 32

10 Prototype Implementation

The implementation uses a request lookup map inside `SharedRoomExamBestFitStrategy`. This map connects an existing `Allocation` back to the original `Request` by using the allo-

cation's `requestId`. This is necessary because `Allocation` itself stores the assigned room and participant count, but it does not directly store the request type.

When checking a room, the strategy follows this logic:

1. Find allocations in the same room with overlapping time slots.
2. If an overlapping allocation belongs to an `ExamRequest`, subtract its `assignedParticipants` from the room capacity.
3. If an overlapping allocation belongs to a non-exam request, block the room.
4. If the request cannot be found, block the room conservatively.
5. Use only rooms with positive remaining capacity.

The strategy first builds temporary selected assignments. It does not immediately create allocation rows. If the total selected capacity is enough, the request is approved and all allocations are committed. If not enough capacity exists, the request is rejected with the reason:

Not enough shared classroom capacity for exam

This prevents partial allocation bugs.

The demo data includes:

- S401: Classroom, capacity 100.
- Request 7: 60 students, Friday 10:00–12:00.
- Request 8: 40 students, Friday 10:00–12:00.

Under `multi_room_exam_best_fit`, Request 7 is approved in S401, but Request 8 is rejected because S401 is blocked.

Under `shared_room_exam_best_fit`, both requests are approved:

Request	Room	Time	Assigned Participants	Par-
7	S401	Friday 10:00–12:00	60	
8	S401	Friday 10:00–12:00	40	

Table 3: Shared-room exam allocation result

The total assigned participants in S401 is:

$$60 + 40 = 100$$

This equals the room capacity, so there is no capacity violation.

11 Summary

Iteration 32 adds a new optional allocation policy for exam scheduling. The new `SharedRoomExamBestFitSt` allows overlapping splittable exams to share the same classroom when enough seating capacity remains.

This iteration keeps previous strategies unchanged. It does not allow normal room sharing, does not mix exams with non-exam bookings, and does not implement global optimization or rescheduling. Instead, it adds a focused strategy that supports a more flexible university exam policy while preserving the existing architecture.

The Strategy Pattern makes both policies available:

- strict room blocking with `multi_room_exam_best_fit`;
- exam-to-exam capacity sharing with `shared_room_exam_best_fit`.

This improves the system's ability to model different real-world exam allocation rules without redesigning the backend.